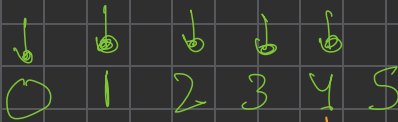
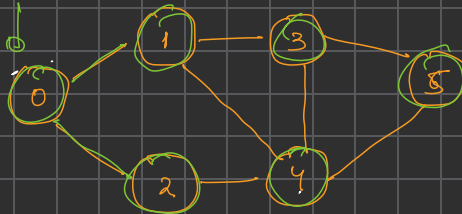




0

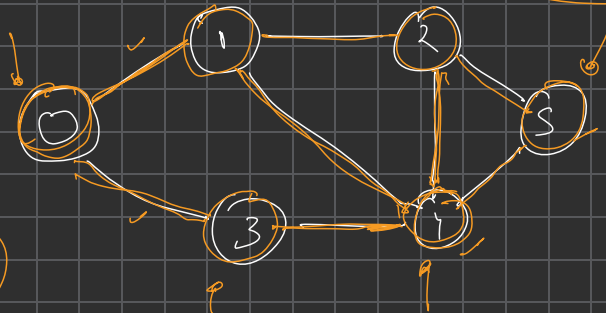
Graph traversal :- BFS, DFS

V ✓
E ✓



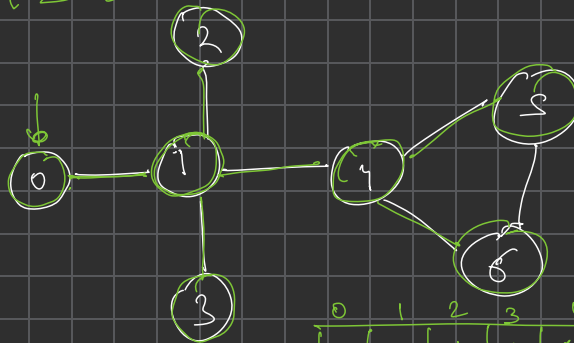
$V + E$

$V + E$
Space complexity $O(V + E)$
 $O(V)$



queue

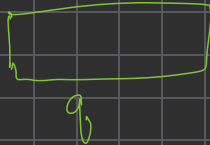
0 1 2 3 4 5 6



BFS
↓
queue

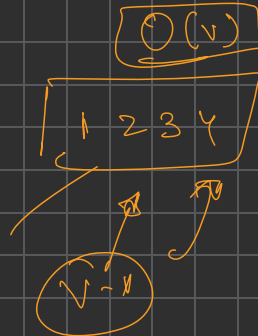
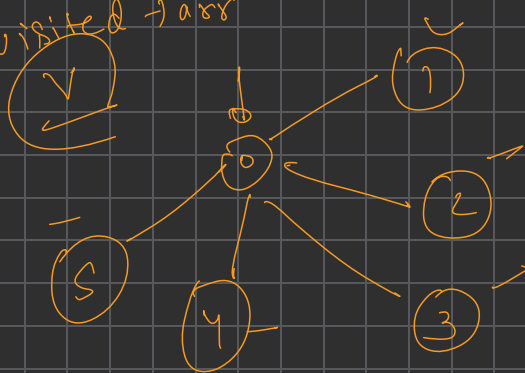
0	1	2	3	4	5	6
1	1	0	0	0	0	0

↑
visited



- ① Create a queue & visited array
- ② push 0 into queue & mark visited of 0 to 1 into visited
- ③ while(!q.empty())
 - ① Take the front element
 - ② Look at its all neighbors
 - ③ push only those neighbors into queue who are not visited yet & mark their neighbor visit to 1

visited $\rightarrow$ arr



adjList $\rightarrow$

$\text{vector} \langle \text{vector} \langle \text{int} \rangle \rangle \text{adj}[v]$



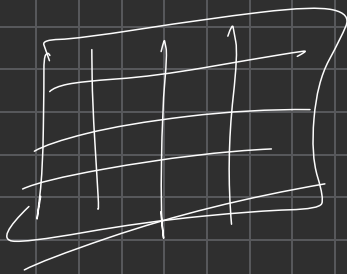
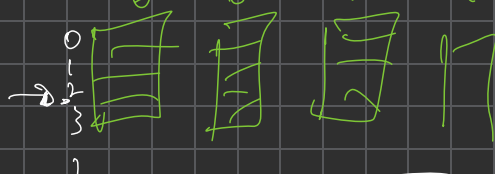
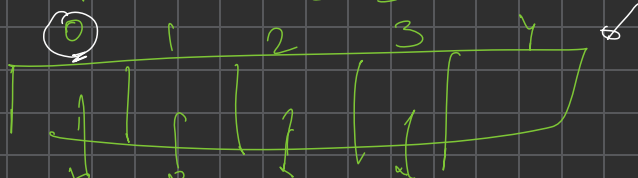
$\text{vector} \langle \text{int} \rangle \text{adj}[5];$

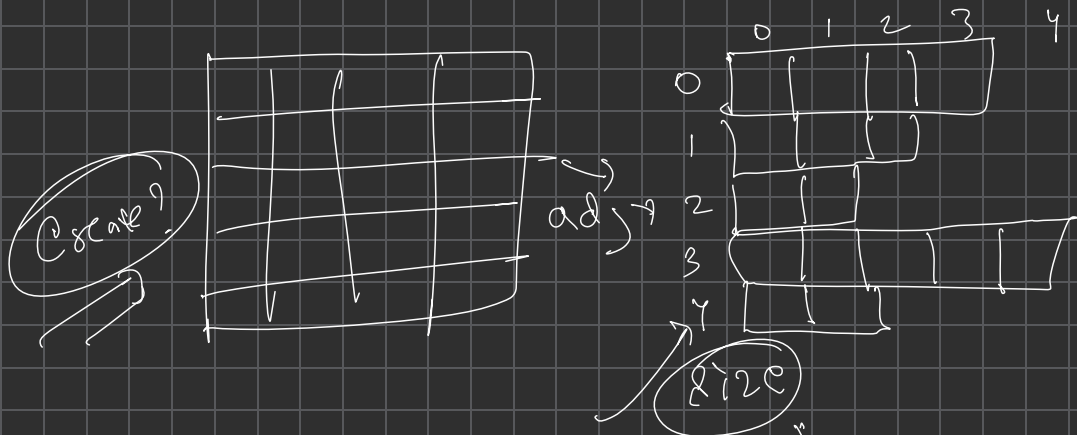
$\text{adj}[0][2];$



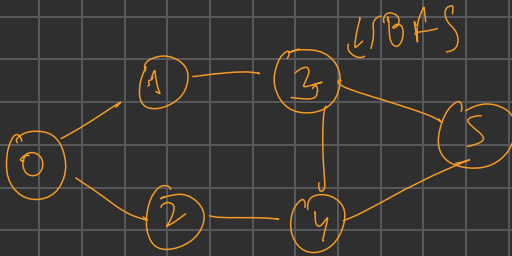
$\text{int arr}[5]$

$[2] [3] [4] [1] [0]$

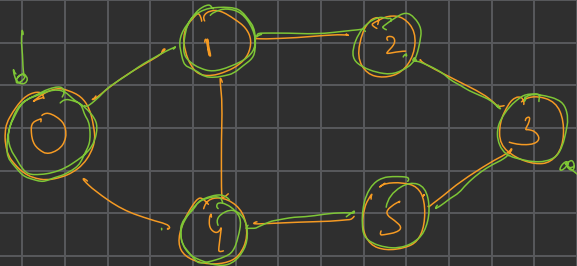




```
for (i=0; adj[3].size(); i++)
    cout << adj[3][i];
```

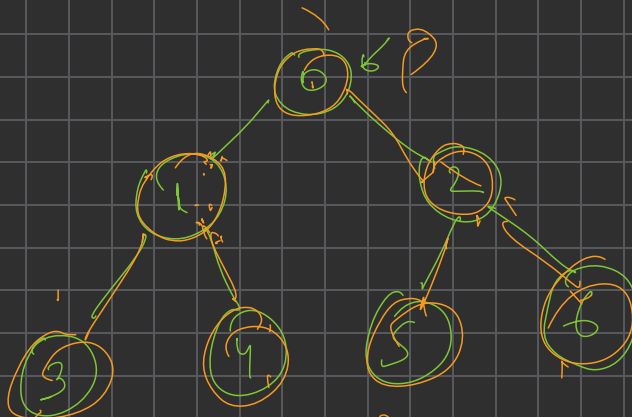


DFS ✓

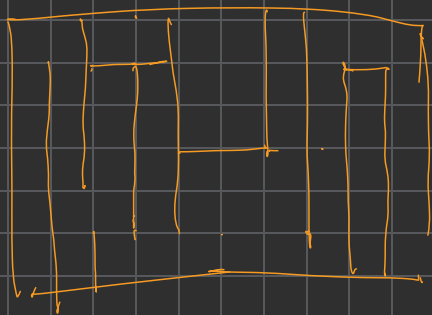


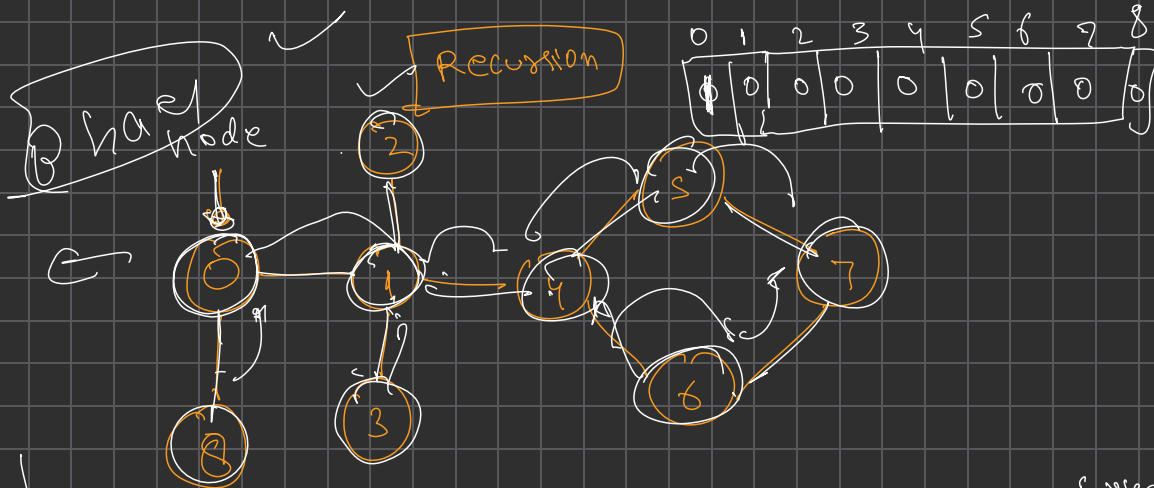
DFS

0 1 2 3 4 5



0 1 3 4 2 5 6





```

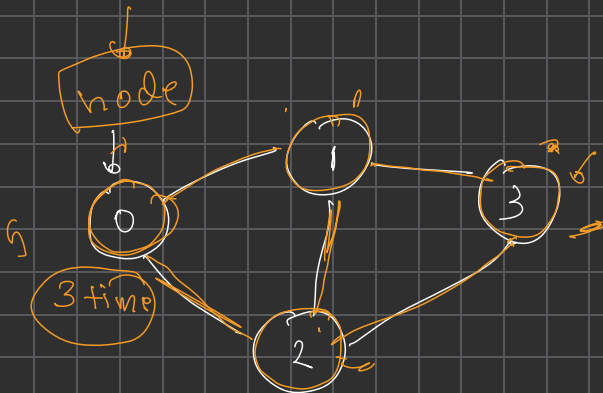
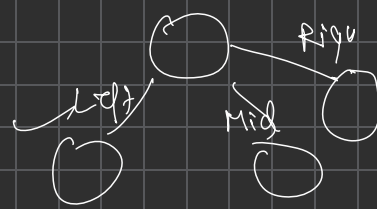
void DFS(vector<int> adj[], node, vector<bool> &visited, vector<int> &result)
{
    visited[node] = 1;
    result.push_back(node);
    for (neib : adj[node])
    {
        if (!visited[neib])
        {
            DFS(adj, neib, visited, result);
        }
    }
}
  
```

cout << root->data

preorder(root->left)

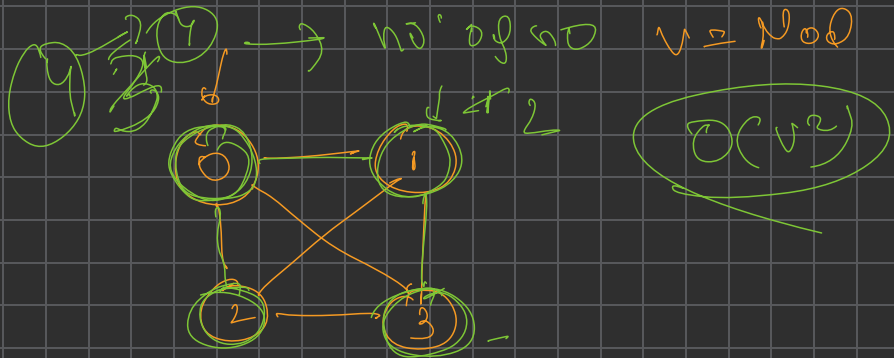
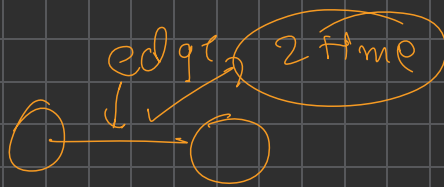
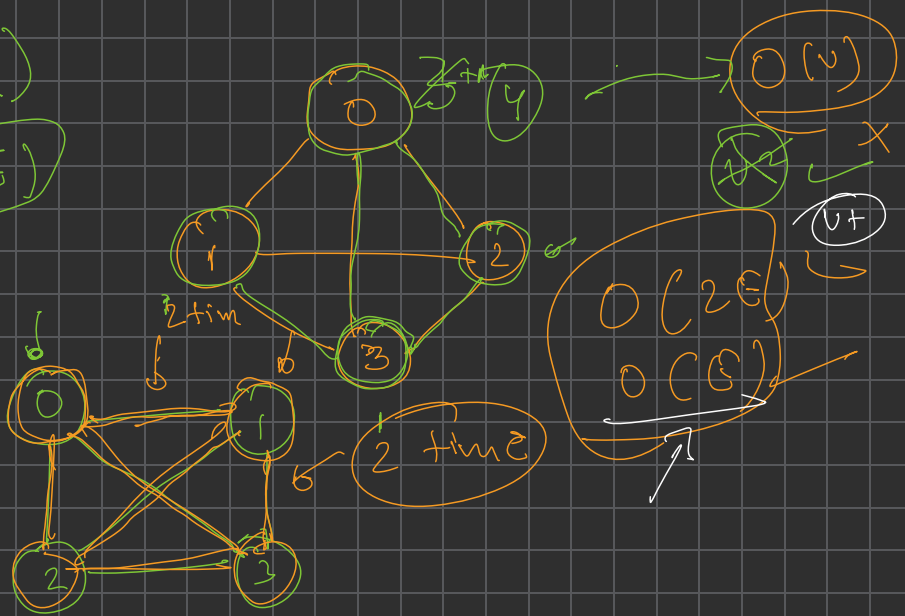
preorder(root->mid)

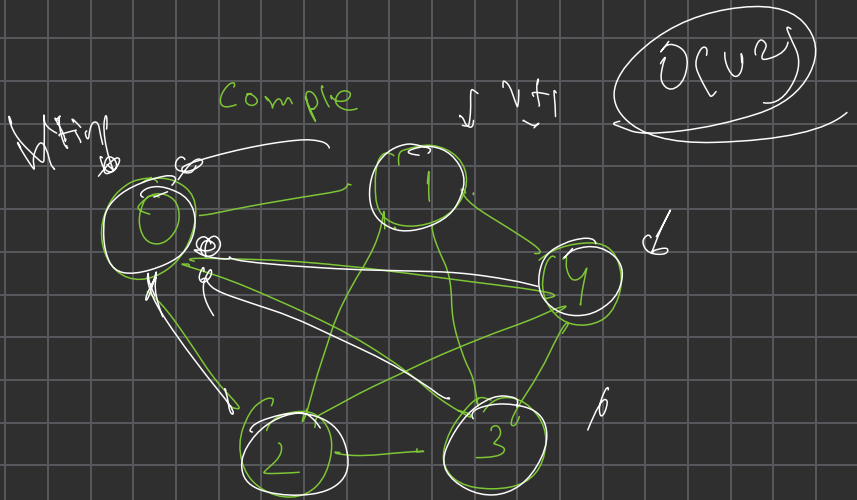
pre (root->right)



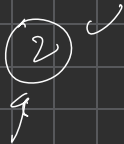
0 (v)

~~$O(V+E)$~~
 $O(E)$





For ($i = 0 ; i < v ; i++$)
 {
 if (!visited[i])
 {
 DFS
 }
 }



Graph
 disconnected
 $O(E)$

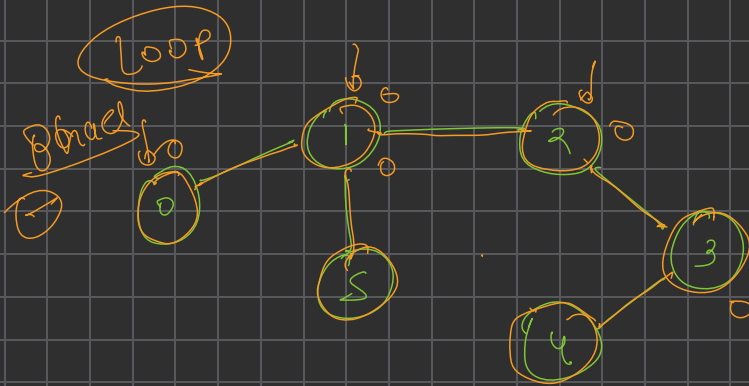
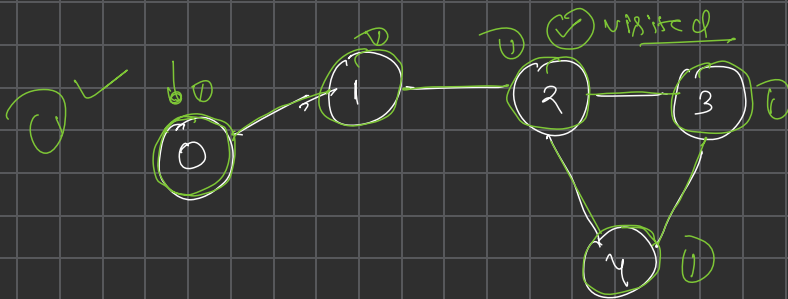
$O(v + E) \rightarrow$ search
 rap

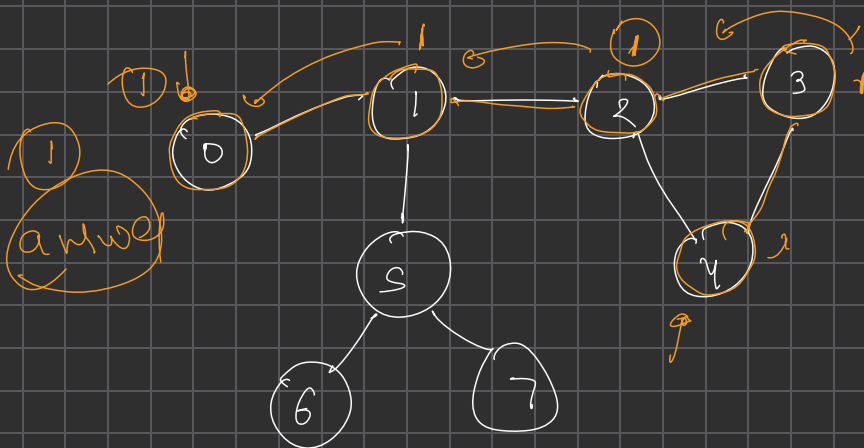
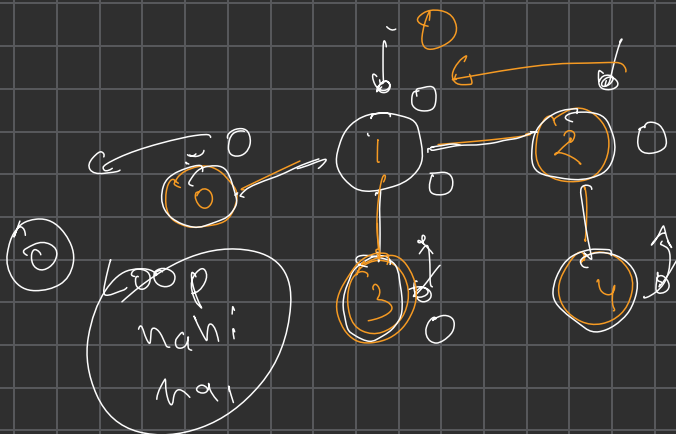
Time $O(V+E)$

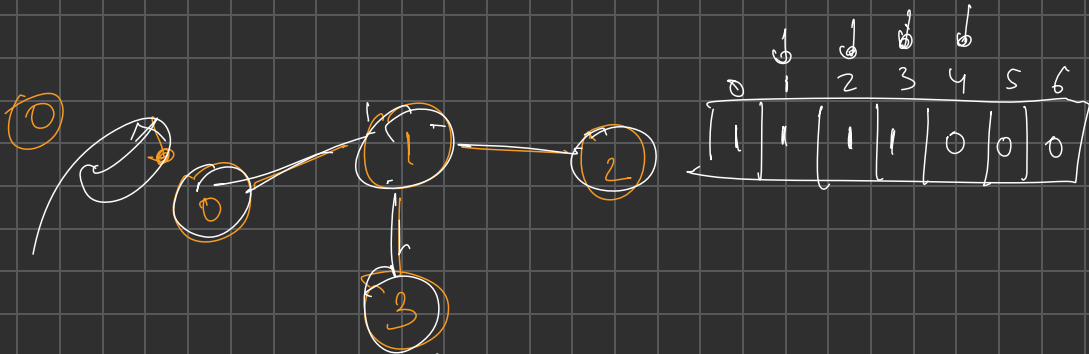
Space $V + V \Rightarrow O(V)$

Recursion
tree









```

foo(i = 0; i < V; i++) {
    if (!visited[i]) {
        if (isInLoop(adj, visited, i, 1)) {
            return 1;
        }
    }
    return 0;
}
  
```

Graph structure (for cycle detection):

- Node 4: points to 5
- Node 5: points to 6 and has a self-loop
- Node 6: points to 4